

— PROJET DESIGN PATTERNS · C#

Usine de drones : assembler par instructions, concevoir par patterns

Du proof of concept naïf à une architecture en 4 design patterns, plus 2 modules complémentaires.

— LE SUJET

Un programme qui pilote l'assemblage de drones

Instruction texte
STOCKS, VERIFY, PRODUCE...

→

Stock & templates

→

Instructions d'assemblage
GET_OUT_STOCK, ASSEMBLE...

— PARCOURS

Trois étapes, trois questions

01 – Naïf

Ça marche — mais un seul fichier, un bloc `if/else` géant.

02 – Patterns

Comment structurer sans sur-ingénierie ? Command, Strategy, Builder, Factory.

03 – Modules

Comment étendre sans casser l'existant ? La réponse : les patterns tiennent leurs promesses.

— PATTERN 1 – COMMAND

Une classe par instruction

```
// étape 1 – un bloc par instruction
if (instr == "STOCKS") {
    PrintStocks(); return;
}
if (instr == "PRODUCE") {
    /* 20 lignes */ return;
}
// ... 6 blocs, un seul fichier
```

```
// étape 2 – un enregistrement
registry.Register("STOCKS",
    new StocksCommand(stocks));

registry.Register("PRODUCE",
    new ProduceCommand(parser, calc,
        stocks));

registry.Dispatch(instruction, arguments);
```

Ajouter RECEIVE/GET_MOVEMENTS en étape 3 : une classe de plus, zéro ligne modifiée dans les 6 commandes existantes.

Une règle par catégorie de drone

```
// Categorization/CategoryRules.cs
class AerienRule : ICategoryRule {
    IsSatisfiedBy(t, catalog) =>
        Any(t.Moves, F, catalog)
        && Get(t.System).HasTag(ThreeD);

}
```

DXF-1	→ Aérien
RDL-1	→ Terrestre
WDS-1	→ Submersible
DYM-1	→ Marin

DroneCategorizer exécute les 4 règles et retourne toutes celles satisfaites — un drone peut cumuler les catégories.

L'ordre d'assemblage, encapsulé

- Générateur avant module principal, déplacement après la coque, système avant assemblage
- Étape 3 : DroneAssemblyBuilder apprend à boucler
générateurs/déplacements multiples → seule

```
> INSTRUCTIONS 1 DXF-1  
PRODUCING DXF-1
```

```
GET_OUT_STOCK 1 Hull_HF1
```

```
INSTALL System_S3D1  
Core_C3D1
```

```
ASSEMBLE TMP1 Hull_HF1  
Generator_GF1
```

```
ASSEMBLE TMP2 TMP1  
Move_MF1
```

```
FINISHED DXF-1
```

Construire et valider un template

```
> ADD_TEMPLATE OK-1, Hull_HS1, Core_C3D1, System_S3D1, Generator_GS1, Move_MS1, Processor_P3D1  
TEMPLATE_ADDED OK-1
```

```
> ADD_TEMPLATE BAD-1, Hull_HF1, Core_CG1, System_SG1, Generator_GG1, Move_MF1, Processor_PG1  
ERROR template `BAD-1` does not match any drone category  
(Aerien/Marin/Terrestre/Submersible)
```

DroneTemplateFactory : rôle de chaque pièce déduit du catalogue, puis validation construction + catégorisation — même chemin pour les 4 drones initiaux et pour ADD_TEMPLATE.

— CHOIX ÉCARTÉS

Ce qu'on n'a pas gardé, et pourquoi

Singleton

pour le stock — une seule instance de `StockRepository` créée dans `Program.cs` et injectée explicitement donne la même unicité, sans état global difficile à isoler.

Abstract
Factory

une seule famille de produits (`DroneTemplate`) — pas de familles cohérentes à faire varier ensemble, donc pas besoin de la variante abstraite.

Observer

pour la traçabilité — un seul abonné (le journal), `StockRepository` enregistre directement chaque mouvement plutôt que notifier un abonné unique.

5.1.1 Réception de stock · 5.1.2 Contraintes de construction

```
> ADD_TEMPLATE BAD-2, Hull_HG1, Core_CG1, System_SG1, Generator_GG1, Move_ML1, Move_MU1,  
Processor_PG1
```

```
ERROR a drone with 2 or more move modules must have  
exactly 2 generators (got 1)
```

```
> RECEIVE 5 Move_MF1, 2 DXF-1  
STOCK_UPDATED
```

1 à 2 générateurs, 1 à 3 modules de déplacement ; ≥ 2 déplacements $\Rightarrow$ 2 générateurs. Règle centralisée dans `ConstructionRules`, réutilisée par la Factory *et* par les modifications `WITH/WITHOUT`.

Modification de drones

- Traçabilité des flux

```
> NEEDED_STOCKS 1 DXF-1 WITH 1 Move_ML1, 1 Generator_GF1
1 DXF-1 :
1 Hull_HF1 · 2 Generator_GF1 · 1 Move_MF1 · 1 Move_ML1

> GET_MOVEMENTS Hull_HF1
-1 Hull_HF1 (PRODUCE)
```

Séparateur ; auto-déTECTÉ dès qu'un mot-clé apparaît ; ancien format virgule toujours accepté. Chaque mutation de StockRepository journalise un motif (PRODUCE, RECEIVE).

— DIFFICULTÉS RENCONTRÉES

Ce qui a résisté

Build

csc.exe de .NET Framework (chemin `$env:WINDIR`, Windows only) → migration vers un **projet dotnet SDK** multiplateforme, cible `net10.0` (seul runtime installé localement)

Dépôt git

`.git` trouvé à la racine du home directory, mêlé à d'autres projets sans rapport → dépôt isolé recréé dans le dossier du projet avec son propre remote

Ambiguïtés

contenu exact d'`ADD_TEMPLATE`, portée de `WITH/WITHOUT` sur `INSTRUCTIONS`, format de `GET_MOVEMENTS`, "toutes les pièces (S)" → chaque choix documenté dans le README plutôt que deviné en silence

— DÉMONSTRATION

En direct

```
> ADD_TEMPLATE MDL-1, Hull_HS1, Core_C3D1, System_S3D1, Generator_GS1, Move_MS1,  
Processor_P3D1
```

```
TEMPLATE_ADDED MDL-1
```

```
> PRODUCE 1 DXF-1 WITH 1 Move_ML1, 1 Generator_GF1  
STOCK_UPDATED
```

```
> GET_MOVEMENTS Hull_HF1  
-1 Hull_HF1 (PRODUCE)
```

— MERCI

Questions ?

Dépôt git — tags [etape-1](#) · [etape-2](#) · [etape-3](#)